Глава 2. Основные управляющие операторы языка C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

2.1 Оператор ветвления

Синтаксис оператора if:

`if (`Логическое_выражение`)` Оператор [`else` Оператор];

Пример:

```
if (a > b)
{
    c = 1;
    k = true;
}
else c = 0;
```

2.2 Оператор выбора

Синтаксис оператора switch:

```
switch (Выражение) {
        case Выражение: {Оператор}
        {case Выражение: {Оператор}}
        [default: Оператор { Оператор }]
}
```

Каждая(!) ветвь должна заканчиваться оператором перехода (break, goto, return).

Пример:

```
switch (k){
        case 1: y = Math.Sin(x); break;
        case 2: y = Math.Cos(x); return;
        default: r = false; break;
}
```

2.3 Операторы цикла

1. Цикл-пока:

   while (Лог_выражение) Оператор;

2. Цикл-до:

   do Оператор **while** (Лог_выражение);

3. Цикл-для:

   for([Инициализация]; [Лог_выражение]; [Модификация])

   [Оператор];

4. Цикл перебора элементов массива или строки и др.(будет рассмотрен далее):

   foreach (Тип Имя **in** Выражение) Оператор;

2.4 Операторы передачи управления

1. Оператор безусловной передачи управления:

```
goto Метка;
...
Метка: Оператор;
```

2. Оператор выхода из конструкции:

```
break;
```

3. Оператор перехода к следующей итерации:

```
continue;
```

2.5 Обработка исключений

Исключение – аварийная ситуация в процессе выполнения программы, например: деление на нуль, переполнение, невозможность преобразования строки в число и т.п.

C# позволяет обрабатывать *синхронные исключения*, которые могут возникнуть в конкретных местах программы, когда какая либо часть программы не смогла выполнить то, что от нее требовалось.

Исключения :

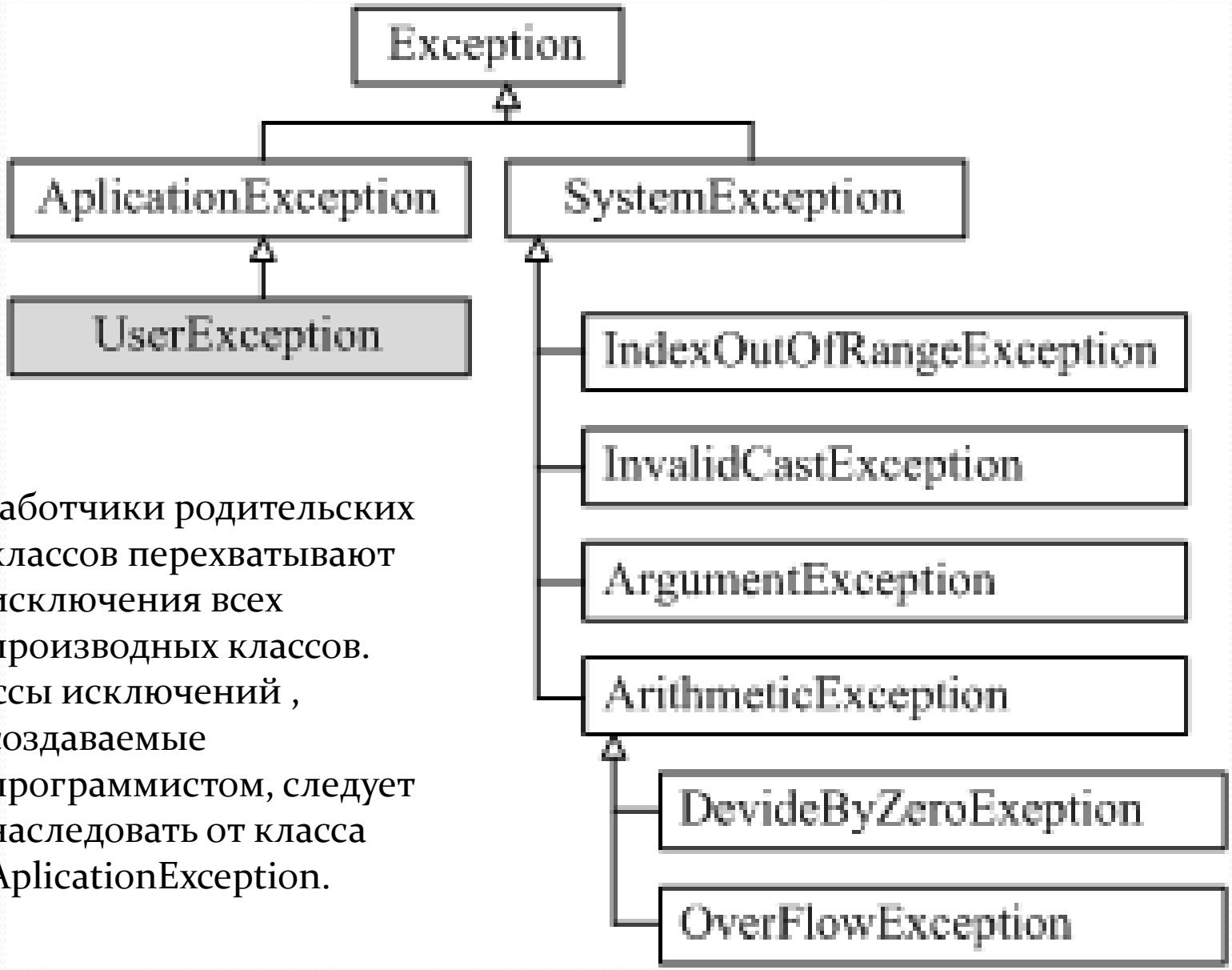
- генерирует среда выполнения программы,
- предусматривает в опасных местах программист, указывая оператор **`throw`**.

В отличие от C++ исключения C# может генерировать среда выполнения программы!!! Поэтому можно перехватить исключение от схем контроля компьютера…

Классы основных стандартных исключений

Имя класса исключения	Описание
`ArithmeticExeption`	Ошибка преобразования или арифметической операции.
`ArrayTypeMismatchExeption`	Попытка сохранить в массиве элемент несовместимого типа.
`DevideByZeroExeption`	Попытка деления на ноль.
`FormatExeption`	Попытка передать в функцию аргумент несовместимого формата.
`IndexOutOfRangeException`	Индекс элемента массива выходит за границы диапазона.
`InvalidCastException`	Ошибка преобразования типа.
`OutOfMemoryException`	Недостаточно памяти для создания нового объекта.
`OverFlowException`	Переполнение при выполнении арифметических операций.
`StackOverFlowException`	Переполнение стека.
`NullReferenceException`	Ссылка на пустой объект.

Фрагмент иерархии классов исключений



Обработчики родительских классов перехватывают исключения всех производных классов.

Классы исключений , создаваемые программистом, следует наследовать от класса AplicationException.

Оператор try

Синтаксис оператора:

`try` Оператор

`catch`([Тип [Имя]]) Оператор

{**`catch`**([Тип [Имя]]) Оператор }

[**`finally`** Оператор]

где

`catch`(**Тип**) – перехватывает исключения заданного и производных (для классов исключений) типов;

`catch`(**Тип Имя**) – позволяет передать в обработчик параметр;

`catch` – обрабатывает любые исключения.

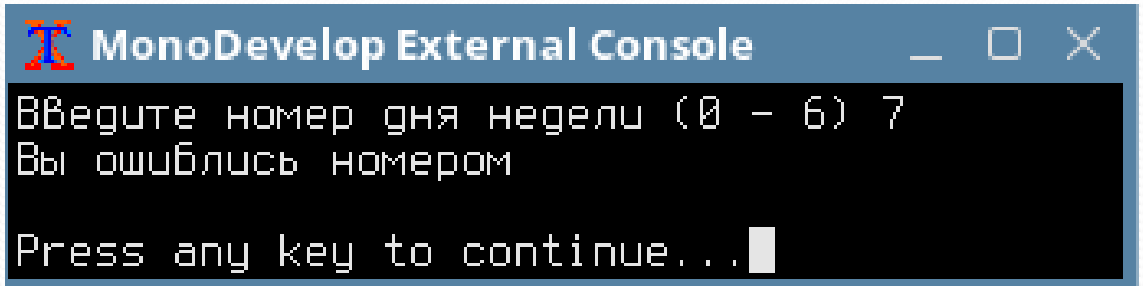
Пример перехвата системного исключения

```
using System;
namespace fff {
  class MainClass {
     public static void Main (string[] args){
        int i = int.Parse(Console.ReadLine ());
        int j = int.Parse(Console.ReadLine());
        try {
            int c = i / j;
        }
        catch (DevideByZeroException) {
            Console.WriteLine ("Osh");                    }
        }
   }
}
```

Пример оператора try

Ex2.01

```
string[] dw = {"Вс","Пн","Вт","Ср","Чт","Пт","Сб" };
Console.Write("Введите номер дня недели (0 - 6) ");
try {
   int i = int.Parse(Console.ReadLine());
   Console.WriteLine("{0}, до Вс {1} дней", dw[i], 6-i);
}
catch (IndexOutOfRangeException) {
   Console.WriteLine("Вы ошиблись номером");
}
catch (FormatException) {
   Console.WriteLine("Введено не число");
}
catch {
   Console.WriteLine("Непонятная ошибка");
}
```



Некоторые свойства класса Exception

`Message` - текстовое описание ошибки (только для чтения)

`Source` - имя объекта, сгенерировавшего ошибку

`TargetSite` - метод, сгенерировавший ошибку

`InnerException` - ссылка на исключение, послужившее причиной текущего

Оператор throw

Формат оператора:

throw [Выражение];

Форма без параметра используется при возобновлении необработанного исключения.

Пример:

```
1)  throw new DevideByZeroExeption(); // генерация исключения
2)  throw;    // возобновление исключения
```

Пример

Ex2.02

```
using System;
namespace Ex2_02 {
class Program
{
    static void F() {
        try { G(); }
        catch (Exception e){
            Console.WriteLine("Exception in F: "+e.Message);
            e = new Exception("F");
            throw e;
        }
    }
    static void G() { throw new Exception( "G" ); }
    static void Main() {
        try { F(); }
        catch ( Exception e){
        Console.WriteLine( "Exception in Main: "+e.Message );
        }
    }
}
```

Операторы и операции checked и unchecked

Оператор(операция) **checked** включает проверку исключений, а оператор(операция) **unchecked** – выключает ее.

1. Синтаксис формы "операция":

checked (Выражение)

unchecked (Выражение)

2. Синтаксис формы "оператор":

checked {Оператор}

unchecked {Оператор}

Примеры:

```
k = checked(a + b);
unchecked{k = a - b};
```